MODULE 09 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Labels, namespaces, ConfigMaps and Secrets

How objects are organised and found, and how configuration gets into Pods.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 9 of 35

admin@sthithapragna.com

Labels **identify and select**. Annotations **describe**. Neither changes what a Pod does.

Labels

- **Key-value pairs** on any object: app=booking-api, tier=backend, env=prod.
- **Not unique.** Many objects share a label; one object has many labels.
- **Selectors find objects by label.** Services, Deployments and kubectl all use them.
- Keys may have a prefix, such as app.kubernetes.io/name. Names and values: up to 63 characters.

> ON THE EXAM
>
> Labels are how loose coupling works: a Service never lists its Pods, it selects them.

Two kinds of selector

KIND	EXAMPLES	WHO SUPPORTS IT
Equality-based	app=web, tier!=cache	Everything, including Services
Set-based	env in (prod,staging), tier notin (cache), !legacy	Deployments, ReplicaSets, Jobs, DaemonSets, kubectl

ON THE EXAM

A Service selector is a plain map: equality only. matchExpressions is where set-based selectors live.

Labels from the command line

```
$ kubectl get pods -l app=booking-api
$ kubectl get pods -l 'env in (prod,staging),tier!=cache'
$ kubectl label pod booking-api-7d9f6c-x2k4p debug=true
$ kubectl get pods --show-labels
```

1. -l filters by an equality selector.
2. Set-based and equality selectors combined; comma means AND.
3. Add a label to a live object.

THE TAKEAWAY

Removing a Pod's app label takes it out of its Service and its ReplicaSet, which then creates a replacement.

Recommended labels

LABEL	QUAYSIDE VALUE
app.kubernetes.io/name	booking-api
app.kubernetes.io/instance	booking-api-prod
app.kubernetes.io/version	2.4.1
app.kubernetes.io/component	api
app.kubernetes.io/part-of	quayside
app.kubernetes.io/managed-by	helm

ON THE EXAM

A shared vocabulary, so dashboards and tools can understand any team's objects.

Labels versus annotations

LABELS

Identify and select

- Used by selectors
- Short values, 63 characters
- Meaningful to Kubernetes
- Example: app=booking-api

ANNOTATIONS

Describe

- Cannot be selected on
- Values can be large
- For people and tools
- Example: git commit, build URL

THE TIEBREAKER

If something selects on it, it is a label. If a person or tool just reads it, it is an annotation.

Namespaces

- **Virtual clusters inside one cluster.** A scope for names: two teams can each have a web Service.
- **A unit for policy:** RBAC permissions, resource quotas and network policies usually apply per namespace.
- **Not a security boundary on their own.** Same nodes, same kernel, open network by default.
- **Deleting a namespace deletes everything in it.**

> ON THE EXAM
>
> Namespaces organise and scope. Isolation needs RBAC, quotas and NetworkPolicy on top.

The four namespaces every cluster starts with

NAMESPACE	HOLDS
`default`	Objects created without a namespace. Fine for learning, poor for production.
`kube-system`	Objects created by Kubernetes itself: CoreDNS, kube-proxy, control plane Pods.
`kube-public`	Readable by everyone, including unauthenticated users. Mostly unused.
`kube-node-lease`	One Lease per node: the heartbeats from module 5.

ON THE EXAM

Four initial namespaces. Exam questions ask for the count and the names.

Namespaced versus cluster-scoped

NAMESPACED

Live inside a namespace

- Pods, Deployments, Services
- ConfigMaps, Secrets, PVCs
- Roles, RoleBindings
- ServiceAccounts, NetworkPolicies

CLUSTER-SCOPED

Belong to the whole cluster

- Nodes, Namespaces
- PersistentVolumes, StorageClasses
- ClusterRoles, ClusterRoleBindings
- CustomResourceDefinitions

REMEMBER

kubectl api-resources --namespaced=false lists every cluster-scoped kind.

Working across namespaces

- **kubectl -n quayside** targets one namespace; **-A** means all namespaces.
- **kubectl config set-context --current --namespace=quayside** changes your default.
- **DNS crosses namespaces:** booking-api.quayside.svc.cluster.local reaches that Service from anywhere.
- A short name like booking-api only resolves within the caller's own namespace.

> ON THE EXAM
>
> Service DNS reads service.namespace.svc.cluster.local: smallest to largest.

Keep configuration **out of the image**, so one image runs in every environment.

A ConfigMap, and two ways to use it

```
apiVersion: v1
kind: ConfigMap
metadata: {name: booking-config}
data:
  LOG_LEVEL: info
  fares.yaml: |
    currency: GBP
---
# inside the booking API's Pod spec:
envFrom: [{configMapRef: {name: booking-config}}]
volumes: [{name: cfg, configMap: {name: booking-config}}]
```

4-7 Simple values, or whole files. Up to 1 MiB, not confidential.

10 As environment variables.

11 As files, once the volume is mounted.

Environment variables versus mounted files

ENVIRONMENT VARIABLES

Read once

- Set when the container starts
- Edit the ConfigMap: nothing changes
- Needs a Pod restart to pick up
- Simple for small settings

MOUNTED VOLUME

Kept up to date

- Files refreshed by the kubelet
- Changes appear after a short delay
- App must reread the file
- Not updated if mounted with subPath

THE TIEBREAKER

Edited a ConfigMap and nothing changed? If it is consumed as env vars, restart the Pods.

Immutable ConfigMaps and Secrets

- **immutable: true** locks the data. Edits are rejected.
- **Protects against accidents:** nobody changes production config in place by mistake.
- **Lighter on the cluster:** kubelets stop watching it for changes.
- **To change it,** create a new one with a new name, and point the Pods at it.

> ON THE EXAM
>
> Versioned, immutable config plus a rollout is a safer pattern than editing config in place.

Secrets: what they are, and are not

- **Same shape as a ConfigMap,** meant for passwords, tokens and keys. Up to 1 MiB.
- **Base64 is encoding, not encryption.** Anyone who can read it can decode it.
- **Stored unencrypted in etcd by default,** unless encryption at rest is configured.
- **Anyone who can create a Pod** in a namespace can mount, and so read, its Secrets.
- **Mounted Secrets live in memory** (tmpfs) on the node, never written to its disk.

> ON THE EXAM
>
> Some material claims Secrets are encrypted by default. They are not. Base64 only.

Base64 is not protection

```
1 $ kubectl get secret db-credentials -o jsonpath='{.data.password}'
2 cGE1NXdvcmQtMjAyNg==
3
4 $ echo 'cGE1NXdvcmQtMjAyNg==' | base64 -d
5 pa55word-2026
```

2 What the API returns: encoded, not encrypted.

5 One command later: the password, in plain text.

THE TAKEAWAY

Protect Secrets with RBAC and encryption at rest, not with base64.

How to actually protect Secrets

LAYER	WHAT IT DOES
Encryption at rest	The API server encrypts Secrets before writing to etcd, ideally with a KMS provider.
RBAC, least privilege	Very few people and ServiceAccounts may get, list or watch Secrets.
External secret stores	Vault or a cloud secret manager, synced in by External Secrets or a CSI driver.
Files, not env vars	Environment variables leak into logs, crash dumps and child processes.
Never plain in Git	Use Sealed Secrets or SOPS if encrypted Secrets must live in a repository.

ON THE EXAM

If asked how to secure Secrets beyond the defaults: encryption at rest plus RBAC.

Secret types

TYPE	HOLDS
`Opaque`	The default: any key-value data.
`kubernetes.io/tls`	A TLS certificate and its private key.
`kubernetes.io/dockerconfigjson`	Registry credentials, used as imagePullSecrets.
`kubernetes.io/basic-auth`	A username and password.
`kubernetes.io/service-account-token`	A legacy long-lived token. Prefer short-lived tokens.

ON THE EXAM

Pulling from a private registry needs a dockerconfigjson Secret named in imagePullSecrets.

> For audits, Quayside wants every Deployment to record the Git commit and the full CI build URL that produced it.
>
> No Service, controller or kubectl selector will ever filter on these values.

Where should this information go?

A Annotations on each Deployment

B Labels on each Deployment

C A ConfigMap for each Deployment

D A namespace named after each commit

◆ PAUSE AND DECIDE

Where should this information go?

A Annotations on each Deployment

Descriptive metadata for people and tools; long values allowed.

B Labels on each Deployment

Nothing selects on it, and a URL breaks the 63-character limit.

C A ConfigMap for each Deployment

ConfigMaps feed apps. This describes the object itself.

D A namespace named after each commit

Namespaces organise workloads, not version history.

THE SIGNAL

'Nothing selects on it' plus long values: an annotation.

> The booking API reads LOG_LEVEL from a ConfigMap, using envFrom. To debug an issue, an engineer changes it from info to debug.
>
> An hour later the running Pods are still logging at info. The ConfigMap shows debug.

What explains this, and what is the fix?

A The kubelet caches ConfigMaps for a day; wait

B ConfigMaps are immutable by default; recreate it

C Env vars are read at start; restart the Pods

D The ConfigMap must be moved into kube-system

◆ PAUSE AND DECIDE

What explains this, and what is the fix?

A The kubelet caches ConfigMaps for a day; wait

Mounted files refresh within minutes; env vars never do.

B ConfigMaps are immutable by default; recreate it

Immutable is opt-in, and the edit clearly succeeded.

C Env vars are read at start; restart the Pods

envFrom is read once. kubectl rollout restart picks up the change.

D The ConfigMap must be moved into kube-system

A Pod can only use ConfigMaps in its own namespace.

THE SIGNAL

Config edited but nothing changed, consumed via env: restart the Pods.

> A security auditor opens a copy of Quayside's etcd backup file and finds the database password, readable after a simple base64 decode.
>
> The password was stored as a Kubernetes Secret, exactly as documented.

Why was the password readable?

A. The backup tool decrypted Secrets while copying them

B. The password was accidentally saved in a ConfigMap

C. The Secret used type Opaque instead of an encrypted type

D. Secrets are just base64 unless encryption at rest is on

◆ PAUSE AND DECIDE

Why was the password readable?

A The backup tool decrypted Secrets while copying them

There was nothing to decrypt: they were never encrypted.

B The password was accidentally saved in a ConfigMap

The scenario says it was a Secret, and base64 is the Secret format.

C The Secret used type Opaque instead of an encrypted type

No Secret type encrypts. Type only describes the data's shape.

D Secrets are just base64 unless encryption at rest is on

The default. Enable encryption at rest, ideally with KMS.

THE SIGNAL

Secret readable in etcd or a backup: encryption at rest was never enabled.

> The payment worker runs in the payments namespace. It must call the booking-api Service, which lives in the quayside namespace.
>
> Both use the default cluster domain, cluster.local.

Which address should the payment worker use?

A booking-api

B booking-api.quayside.svc.cluster.local

C quayside.booking-api.svc.cluster.local

D booking-api.payments.svc.cluster.local

◆ PAUSE AND DECIDE

Which address should the payment worker use?

A booking-api

A short name resolves in the caller's own namespace: payments.

B booking-api.quayside.svc.cluster.local

Service, then namespace, then svc and the cluster domain.

C quayside.booking-api.svc.cluster.local

Namespace and Service are the wrong way round.

D booking-api.payments.svc.cluster.local

Looks in the caller's namespace, where no such Service exists.

THE SIGNAL

Crossing namespaces: service.namespace.svc.cluster.local.

Module 9 in five lines

- **Labels select, annotations describe.** Services use equality selectors only.
- Four initial namespaces: **default, kube-system, kube-public, kube-node-lease**.
- Cross-namespace DNS: **service.namespace.svc.cluster.local**.
- ConfigMap as **env vars: read once**; as **files: refreshed**.
- Secrets are **base64, not encrypted**. Add encryption at rest and RBAC.

You have met a dozen kubectl commands.
Next: **kubectl fluency**, all in one place.